

1) 8-Puzzle Problem Using Python

Aim

To develop a Python program to solve the 8-puzzle problem using the A* search algorithm.

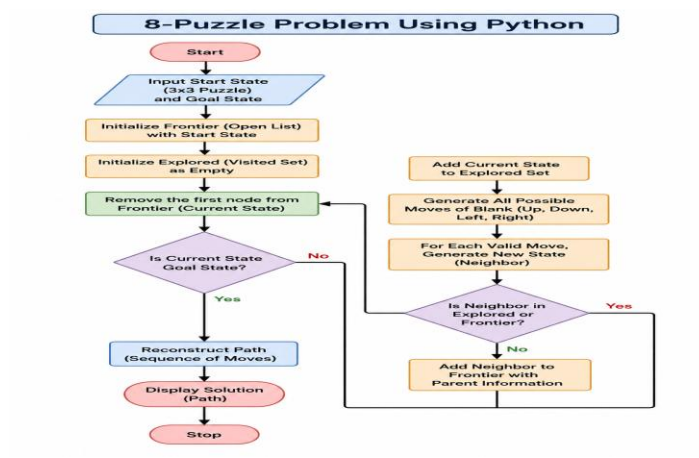
Objectives

- Understand the state-space representation of the 8-puzzle problem.
- Apply the A* search algorithm with Manhattan distance heuristic.
- Find the optimal sequence of moves from the initial state to the goal state.
- Demonstrate informed search in Artificial Intelligence.

Algorithm

1. Define the initial state and goal state.
2. Use a priority queue (min-heap) to store puzzle states.
3. Calculate the Manhattan distance heuristic for each state.
4. Expand the state with the lowest $f(n)=g(n)+h(n)$.
5. Generate all valid neighboring states by moving the blank tile.
6. Ignore already visited states.
7. Repeat until the goal state is reached.
8. Display the sequence of moves.

Flow Chart



Python Program

```
import heapq

goal = ((1,2,3),(4,5,6),(7,8,0))

def heuristic(state):
    dist = 0
    for i in range(3):
        for j in range(3):
            val = state[i][j]
            if val != 0:
                x = (val-1)//3
                y = (val-1)%3
                dist += abs(i-x)+abs(j-y)
    return dist

def neighbors(state):
    state = [list(r) for r in state]
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                x,y=i,j
    moves=[(-1,0),(1,0),(0,-1),(0,1)]
    result=[]
    for dx,dy in moves:
        nx,ny=x+dx,y+dy
        if 0<=nx<3 and 0<=ny<3:
            new=[row[:] for row in state]
            new[x][y],new[nx][ny]=new[nx][ny],new[x][y]
            result.append(tuple(tuple(r) for r in new))
    return result

def astar(start):
    pq=[]
    heapq.heappush(pq,(heuristic(start),0,start,[start]))
    visited=set()
    while pq:
        f,g,state,path=heapq.heappop(pq)
        if state==goal:
            return path
```

```

        if state in visited:
            continue
        visited.add(state)
        for nxt in neighbors(state):
            if nxt not in visited:
                heapq.heappush(pq,(g+1+heuristic(nxt),g+1,nxt,path+[nxt]))
    return None

start=((1,2,3),(4,0,6),(7,5,8))
solution=astar(start)

for step,s in enumerate(solution):
    print("Step",step)
    for row in s:
        print(row)
    print()

```

Sample Output

```

Output
Step 0
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 2
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

=== Code Execution Successful ===

```

Result

Thus, the Python program to solve the 8-puzzle problem using the A* search algorithm was implemented successfully, and the optimal solution path was obtained.

Conclusion

The A* search algorithm efficiently solves the 8-puzzle problem by combining path cost and heuristic estimation. Using the Manhattan distance heuristic reduces unnecessary state exploration and helps find the shortest solution path effectively.

2) 8 – QUEEN PROBLEM USING PYTHON

Aim

To develop a Python program to solve the 8-Queen Problem using the Backtracking Algorithm.

Objectives

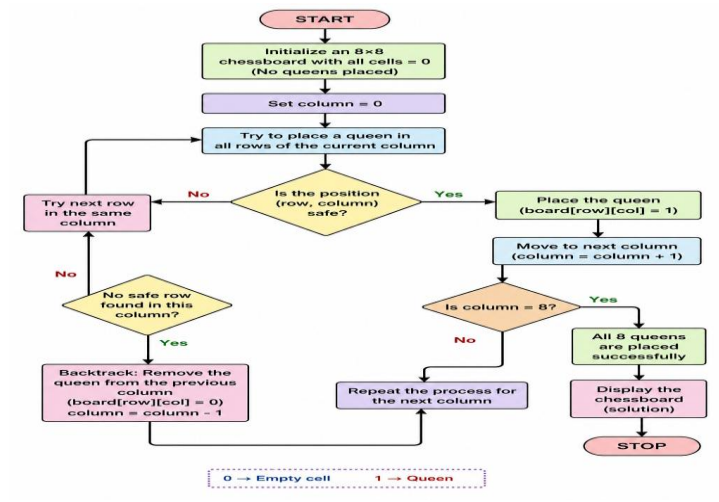
- To understand the 8-Queen problem in Artificial Intelligence.
- To apply the Backtracking technique for finding a valid solution.
- To place eight queens on an 8×8 chessboard so that no two queens attack each other.
- To display a valid arrangement of queens on the chessboard.

Algorithm

1. Start the program.
2. Create an empty 8×8 chessboard.
3. Begin placing queens from the first column.
4. Check whether placing a queen in the current row is safe.
5. If the position is safe, place the queen.
6. Move to the next column and repeat the process.
7. If no safe position exists, remove the previously placed queen (Backtracking).
8. Continue until all eight queens are placed.
9. Display the chessboard with the queen positions.

10. Stop the program.

Flow Chart



Python Program

N = 8

```
def printSolution(board):
```

```
    for i in range(N):
```

```
        for j in range(N):
```

```
            print(board[i][j], end=" ")
```

```
        print()
```

```
def isSafe(board, row, col):
```

```
    for i in range(col):
```

```
        if board[row][i] == 1:
```

```
            return False
```

```
    i = row
```

```
j = col
while i >= 0 and j >= 0:
    if board[i][j] == 1:
        return False
    i -= 1
    j -= 1
```

```
i = row
j = col
while i < N and j >= 0:
    if board[i][j] == 1:
        return False
    i += 1
    j -= 1
```

```
return True
```

```
def solve(board, col):
    if col >= N:
        return True

    for i in range(N):
        if isSafe(board, i, col):
            board[i][col] = 1
```

```
        if solve(board, col + 1):
            return True

        board[i][col] = 0

    return False

board = [[0] * N for i in range(N)]

if solve(board, 0):
    print("Solution Found")
    printSolution(board)
else:
    print("No Solution Exists")
```

Sample Output

```
Output
Solution Found
1 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0

=== Code Execution Successful ===
```

Result

Thus, the Python program to solve the 8-Queen Problem using the Backtracking Algorithm was implemented successfully, and a valid arrangement of eight queens was obtained.

Conclusion

The Backtracking Algorithm efficiently solves the 8-Queen Problem by placing queens one column at a time and backtracking whenever a conflict occurs. This approach guarantees a valid solution while reducing unnecessary computations, making it an effective Artificial Intelligence search technique.